

Relazione modello fisico chitarra

Sommario

Nucleo di funzionamento:	3
1- Eccitazione (Plettrata / Dito):	3
Hardness: filtro Passa-Basso	3
Ampiezza: la posizione del suono	4
Immissione nel sistema	4
2 - Linea di ritardo e simulazione della corda:	5
Calcolo della lunghezza (pitch)	5
Movimento dell'onda (avanzamento nel buffer)	5
3 - Filtro Passa-Basso (Damping):	5
4 - Feedback (Sustain):	6
Coefficiente di attenuazione	6
Uscita audio	6
Schema a blocchi:	7
Catena degli effetti:	8
1 - Distorsione (Soft-Clipping Iperbolico):	8
2 - Delay (Ring Buffer e Feedback):	8
3 - Riverbero:	9
4 - Master e Metering:	10
Architettura e Thread Safety	11
Gestione dei parametri: da UI ad Audio (APVTS)	11
Gestione dell'animazione:	11
Interfaccia Grafica e User Experience	13
Animazione Fisica delle Corde (Visual Karplus-Strong):	13
Sistema di Preset:	13
Scalabilità Dinamica:	14

Nucleo di funzionamento:

1- Eccitazione (Plettrata / Dito)

Nella funzione `generateExcitation()` della classe `StringSynthesiser` viene generato un valore casuale tra -1 e 1 per ogni campione iniziale.

Questo ciclo genera una scarica di rumore bianco, punto di partenza dell'algoritmo.

Ogni campione varia in modo casuale dal precedente, e nello specifico maggiore è la differenza tra il valore di un dato campione e di quello precedente, più il segnale dovrà variare rapidamente in intervalli brevissimi di tempo.

Questo porta dunque ad un segnale carico di alte frequenze, che restituisce un suono secco e "metallico".

```
void generateExcitation() {
    float lastSample = 0.0f;
    float maxVal = 0.0f;

    for (size_t i = 0; i < currentDelayLength; ++i)
    {
        //genera rumore bianco casuale tra -1 e 1 (ogni campione è indipendente dal precedente)
        float noise = (juce::Random::getSystemRandom().nextFloat() * 2.0f) - 1.0f;
```

Hardness: filtro Passa-Basso

Si calcola una media pesata (filtro IIR del primo ordine) tra il nuovo campione casuale (noise) e il campione precedente (`lastSample`).

Si va di fatto a limitare matematicamente quanto un campione può al più variare dal precedente (dando più peso al precedente il valore risultante è costretto a non allontanarsi numericamente di molto).

Questo approccio, ponendo una Hardness bassa, riduce conseguentemente le alte frequenze nel segnale (dovute a cambi repentini nei valori dei campioni dello stesso).

- Più `currentHardness` è bassa (più tende a 0), più il suono risulterà carente in alte frequenze, simulando la pizzicata del dito
- In maniera speculare, maggiore è la `currentHardness` (più vicina all'1), più il suono sarà percepito come metallico.

Come accennato, proprio grazie all'abbondanza di alte frequenze, si simulerà l'effetto del plettro sulle corde.

```
//media pesata tra il campione nuovo e il precedente
//dove 1 = plettro e 0 = dito
//un po' come un filtro passa-basso di primo ordine
//dove più l'hardness è bassa e più taglia gli acuti
float shaped = noise * currentHardness + lastSample * (1.0f - currentHardness);
```

Ampiezza: la posizione del suono

Quando la corda viene suonata, si calcola l'ampiezza generale "amplitude". La formula matematica utilizzata fa sì che pizzicando la corda esattamente al suo centro si ottenga la massima vibrazione (ampiezza = 1), mentre pizzicandola verso le estremità (vicino al ponte della chitarra) la vibrazione risulti progressivamente minore, rispecchiando il reale comportamento acustico.

```
void stringPlucked(float pluckPosition)
{
    if (doPluckForNextBuffer.compareAndSetBool(1, 0))
        amplitude = std::sin(juce::MathConstants<float>::pi * pluckPosition);
}
```

Immissione nel sistema

Si prende il burst di rumore filtrato (excitationSample) e lo si riscrive nella linea di ritardo (delayLine), moltiplicandolo per l'ampiezza precedente.

```
void exciteInternalBuffer()
{
    generateExcitation();
    for (size_t i = 0; i < currentDelayLength; ++i)
        delayLine[i] = excitationSample[i] * (float)amplitude;
}
```

2 - Linea di ritardo e simulazione della corda

La linea di ritardo è rappresentata da `std::vector<float> delayLine`. Il tempo che l'onda impiega a percorrere il buffer determina la frequenza della nota.

Calcolo della lunghezza (pitch)

Dividendo la frequenza di campionamento (44100 Hz) per la frequenza della nota, si ottengono quanti campioni sono presenti in un singolo ciclo dell'onda. Questa è, effettivamente, la lunghezza della corda virtuale in quel momento specifico.

```
void setFrequency(double newFrequencyInHz)
{
    currentDelayLength = (size_t)juce::roundToInt(fs / newFrequencyInHz);
    currentDelayLength = juce::jlimit((size_t)2, maxDelayLength, currentDelayLength);
    pos = 0; // reset posizione per evitare artefatti
    generateExcitation();
}
```

Movimento dell'onda (avanzamento nel buffer)

Durante la generazione dell'audio, si avanza di un punto alla volta nel buffer. L'operatore "%" assicura che, una volta terminato il buffer, si ricominci dall'inizio, creando l'effetto "loop" del Ring Buffer.

La frazione di rumore iniziale, venendo ripetuta identica a se stessa centinaia di volte al secondo, smette di essere percepita come "rumore" e si trasforma in un'onda periodica. La velocità con cui questo loop si ripete crea la percezione dell'intonazione (pitch) della nota.

```
void generateAndAddData(float* outBuffer, int numSamples)
{
    if (doPluckForNextBuffer.compareAndSetBool(0, 1))
        exciteInternalBuffer();

    //damping
    float dampCoeff = currentDamping * 0.5f;

    for (int i = 0; i < numSamples; ++i)
    {
        auto nextPos = (pos + 1) % currentDelayLength;

        //filtraggio damping
        float filtered = delayLine[pos] * (1.0f - dampCoeff) + delayLine[nextPos] * dampCoeff;

        //sustain: feedback gain tra 0.9f e 0.999f
        float feedbackGain = 0.9f + currentSustain * 0.099f;

        delayLine[nextPos] = filtered * feedbackGain;

        outBuffer[i] += delayLine[pos];
        pos = nextPos;
    }
}
```

3 - Filtro Passa-Basso (Damping)

All'interno del ciclo for di `generateAndAddData`, si applica la simulazione delle perdite fisiche del materiale. Come per la Hardness iniziale, si usa un filtro Passa-Basso per smussare l'onda mentre viaggia. Si prende il campione attuale (`pos`) e quello successivo (`nextPos`) e si

fa una media pesata. Più alto è il `dampCoeff`, più la corda perderà il tono metallico (le alte frequenze) velocemente ad ogni giro del buffer.

4 - Feedback (Sustain)

L'ultimo passaggio è la perdita di energia complessiva.

Coefficiente di attenuazione

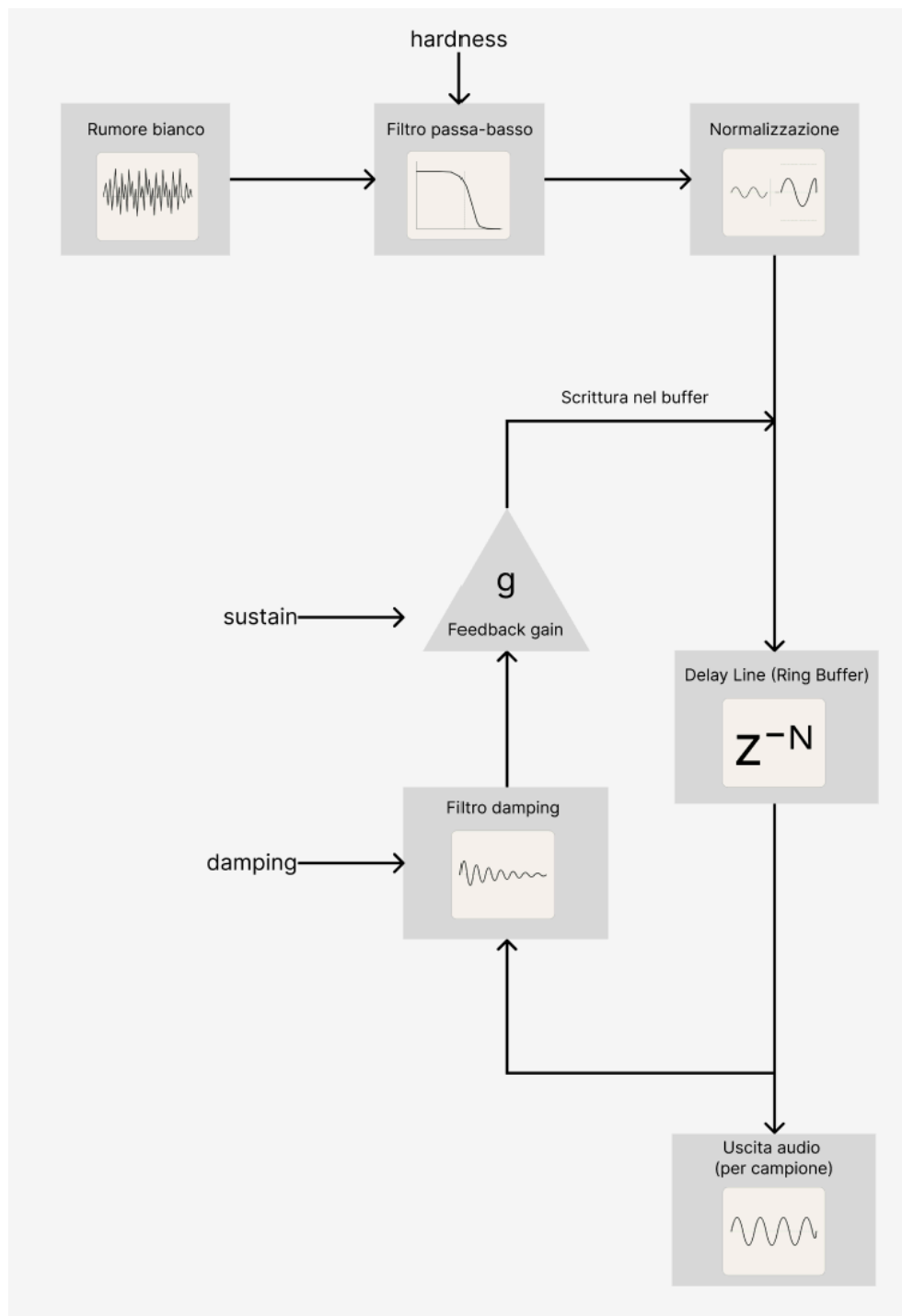
Il segnale filtrato (filtered) viene moltiplicato per `feedbackGain`. Anche con il sustain al massimo, il gain totale risulta essere uguale a 0.999. Questo garantisce che ad ogni giro del loop il segnale si riduca, creando il decadimento naturale del suono e impedendo al plugin di esplodere.

Uscita audio

Infine, si prende il campione dalla linea di ritardo e si somma al buffer audio in uscita verso le casse.

Nota: si usa += invece di = in modo da permettere il suono di più corde contemporaneamente, sommando insieme i suoni risultanti.

Schema a blocchi:



Catena degli effetti:

Terminata la sintesi fisica dell'onda sonora nella classe `StringSynthesiser`, il segnale Dry viene iniettato nel buffer principale dell'Audio Processor. Gli effetti sono stati progettati a catena, per evitare conflitti, e ciascuno di essi è dotato di un parametro di Bypass (On / Off) per ottimizzare il carico della CPU quando l'effetto non è richiesto (e permettere la scelta all'utente di utilizzare o meno un determinato effetto).

1 - Distorsione (Soft-Clipping Iperbolico):

Per simulare il comportamento non-lineare delle valvole di un amplificatore per chitarra, è stata implementata una tecnica di Waveshaping utilizzando la funzione matematica della tangente iperbolica (`std::tanh`).

```
// (Amplifico al cubo per ottenere un effetto di drive più marcato)
float appliedDrive = currentDrive * currentDrive * currentDrive;

#pragma endregion

// Ciclo per ogni canale...
for (int ch = 0; ch < buffer.getNumChannels(); ++ch)
{
    // Accedo tramite puntatore al buffer del canale
    auto* channelData = buffer.getWritePointer(ch);

    // Quindi applico la distorsione sample per sample nel buffer del canale
    for (int numSample = 0; numSample < buffer.getNumSamples(); ++numSample)
    {
        // Soft Clipping via tangente iperbolica.
        channelData[numSample] = std::tanh(channelData[numSample] * appliedDrive) * currentGain;
    }
}
```

Viene usata la tangente iperbolica in quanto l'audio, mappando qualsiasi valore di ingresso nel range $[-1, 1]$, produce un soft-clipping. Questo permette la generazione di armoniche dispari. Per rendere l'escursione più aggressiva, il valore del parametro Drive viene elevato al cubo (`appliedDrive`). Infine, poiché la distorsione incrementa l'energia del segnale, il risultato viene moltiplicato per `currentGain` per compensare il volume e mantenere il suono bilanciato.

2 - Delay (Ring Buffer e Feedback):

Successivamente, l'audio entra nel modulo di Delay. Questo effetto si basa sulla memorizzazione dell'audio in un buffer secondario (`delayBuffer`) che funge da "nastro magnetico" circolare.


```

// Calcoliamo a quanti "campioni" corrisponde il ritardo in secondi
int delayLengthInSamples = (int)(timeInSeconds * currentSampleRate);
int delayBufferLength = delayBuffer.getNumSamples();

// Ciclo sui canali (L e R)
for (int ch = 0; ch < buffer.getNumChannels(); ++ch)
{
    auto* channelData = buffer.getWritePointer(ch);
    // Assicuriamoci di leggere il canale giusto anche nel delayBuffer
    auto* delayData = delayBuffer.getWritePointer(ch % delayBuffer.getNumChannels());

    // Copiamo la posizione di scrittura per questo specifico canale
    int localWritePosition = delayWritePosition;

    for (int i = 0; i < buffer.getNumSamples(); ++i)
    {
        // Calcoliamo la testina di lettura (indietro nel tempo rispetto alla scrittura)
        int readPosition = localWritePosition - delayLengthInSamples;
        if (readPosition < 0)
            readPosition += delayBufferLength; // Se andiamo sotto zero, facciamo il giro del ring buffer

        // Prendiamo il campione ritardato (l'eco)
        float delayedSample = delayData[readPosition];

        // Scriviamo nel "nastro" il suono attuale + l'eco attenuato (Feedback)
        delayData[localWritePosition] = channelData[i] + (delayedSample * feedback);

        // Aggiungiamo l'eco al suono originale in uscita (Mix al 50%)
        channelData[i] += delayedSample * 0.5f;

        // Avanziamo la testina di scrittura locale di 1 step
        localWritePosition++;
        if (localWritePosition >= delayBufferLength)
            localWritePosition = 0;
    }

    // Finito il blocco audio, aggiorniamo la posizione di scrittura globale per il prossimo giro
    delayWritePosition += buffer.getNumSamples();
    delayWritePosition %= delayBufferLength;
}
else
{
    // Se il delay è spento, assicuriamoci di avanzare comunque la posizione di scrittura per mantenere la coerenza del buffer
    delayWritePosition += buffer.getNumSamples();
    delayWritePosition %= delayBuffer.getNumSamples();
}
}

```

L'algoritmo usa due testine logiche: una di lettura e una di scrittura. Quella di lettura si trova su `delayLengthInSamples`. Quando scende sotto lo zero, viene aggiunto un offset pari alla grandezza totale del buffer per farlo ripartire dalla fine, chiudendo l'anello. Le due testine, per la lunghezza del segnale, si inseguono, e quella di lettura andrà di fatto a leggere il segnale trascritto da quella di scrittura, dopo un certo tempo (`delayTime`). Il parametro Feedback determina quanta parte del segnale ritardato viene re-iniettata nella testina di scrittura, decidendo il numero di echi prima che il suono si estingua.

3 - Riverbero:

Il progetto sfrutta il modulo `Juce::Reverb` integrato nel framework.

```

if (revOnParameter->load() >= 0.5f)
{
    // Aggiorniamo i parametri del riverbero leggendo i valori dalle manopole
    float mix = revMixParameter->load() / 100.0f;

    reverbParams.roomSize = revSizeParameter->load() / 100.0f; // Da 0.0 (stanza piccola) a 1.0 (stanza grande)
    reverbParams.damping = 0.5f;
    reverbParams.width = 1.0f; // Massima ampiezza stereo

    // Calcolo Dry/Wet: se Mix è 0, si sente solo la chitarra; se Mix è 1, si sente solo il riverbero
    reverbParams.dryLevel = 1.0f - mix;
    reverbParams.wetLevel = mix;

    reverb.setParameters(reverbParams);

    // Appliciamo il riverbero
    if (buffer.getNumChannels() >= 2)
    {
        float* leftChannel = buffer.getWritePointer(0);
        float* rightChannel = buffer.getWritePointer(1);

        // funzione standard di JUCE per processare un buffer stereo con il riverbero
        reverb.processStereo(leftChannel, rightChannel, buffer.getNumSamples());
    }
}

```

Prima di invocare il processore del riverbero, i valori percentuali forniti dall'interfaccia utente vengono mappati nel range lineare [0.0, 1.0]. In particolare, il parametro Rev Mix gestisce il

bilanciamento Dry / Wet in modo complementare: all'aumentare del livello wet (`wetLevel`), diminuisce il segnale asciutto (`dryLevel`), mantenendo il suono in uscita coerente. La funzione `processStereo` applica infine il ritardo.

4 - Master e Metering:

L'ultimo passaggio prima di consegnare il buffer audio alla DAW è l'applicazione del Master Volume.

```
#pragma region Applicazione Master Volume
float masterVolume = masterVolumeParameter->load() / 100.0f;
//Metodo per applicare il volume a tutto il buffer
buffer.applyGain(masterVolume);
#pragma endregion
```

Si utilizza la funzione ottimizzata `applyGain()`, che sfrutta le istruzioni vettoriali della CPU per moltiplicare l'intero blocco di memoria in un'unica operazione (invece di iterare con cicli for, più dispendiosi).

Architettura e Thread Safety

L'elaborazione del segnale all'interno del metodo `processBlock` avviene in un thread dedicato ad altissima priorità (Audio Thread). Per evitare artefatti dovuti all'elaborazione del suono in tempo reale, la comunicazione tra interfaccia grafica (UI Thread) e il processore audio è stata progettata seguendo paradigmi lock-free.

Gestione dei parametri: da UI ad Audio (APVTS)

Al fine di garantire una comunicazione sicura tra i controlli dell'interfaccia utente e il motore di elaborazione del segnale, il sistema implementa la classe `AudioProcessorValueTreeState` (APVTS). Tale struttura permette di sincronizzare i parametri in tempo reale durante il calcolo audio.

Tuttavia, leggere il valore direttamente dall'APVTS all'interno del `processBlock` comporterebbe una ricerca testuale basata su stringhe (lookup), un'operazione troppo lenta per l'Audio Thread. La soluzione implementata fa uso di puntatori atomici inizializzati una sola volta nel costruttore:

```
#pragma region APVTS parameters
// Prendo i riferimenti ai parametri di controllo dell'effettistica dalla APVTS
// (da usare in processBlock) (e quindi da dereferenziare)
driveParameter = apvts.getRawParameterValue("drive");
gainParameter = apvts.getRawParameterValue("gain");
hardnessParameter = apvts.getRawParameterValue("hardness");
dampingParameter = apvts.getRawParameterValue("damping");
sustainParameter = apvts.getRawParameterValue("sustain");
revMixParameter = apvts.getRawParameterValue("revMix");
revSizeParameter = apvts.getRawParameterValue("revSize");
delayTimeParameter = apvts.getRawParameterValue("delayTime");
delayFbParameter = apvts.getRawParameterValue("delayFb");
masterVolumeParameter = apvts.getRawParameterValue("masterVolume");
delayOnParameter = apvts.getRawParameterValue("delayOn");
distOnParameter = apvts.getRawParameterValue("distOn");
revOnParameter = apvts.getRawParameterValue("revOn");
#pragma endregion
```

Attraverso l'invocazione di `getRawParameterValue()`, il sistema recupera un puntatore diretto a un'istanza di `std::atomic<float>`. Tale valore viene acquisito nel `processBlock` impiegando il metodo `load()`: una procedura che assicura la corretta sincronizzazione dei dati.

Gestione dell'animazione:

La problematica speculare, ovvero la trasmissione dati dal thread audio all'interfaccia utente, emerge nel momento in cui la ricezione di un messaggio MIDI deve attivare la rappresentazione visiva della corda vibrante. Non è possibile utilizzare procedure grafiche come `repaint()` direttamente all'interno del `processBlock`. Per ovviare a tale limite, l'architettura utilizza variabili atomiche.

All'interno della logica del processore, il monitoraggio dello stato di ciascuna corda viene affidato a due specifiche strutture di array atomici:

```
// Atomic per gestire in maniera thread-safe le modifiche alla UI da parte del processBlock
std::atomic<bool> uiStringWasPlucked[numStrings];
std::atomic<float> uiPluckPosition[numStrings];
```

```
// Comunico tramite le variabili atomic (maniera thread-safe) alla UI
// che la corda i è stata pizzicata e qual è la posizione del tasto.
uiPluckPosition[i].store(position);
uiStringWasPlucked[i].store(true);
```

Aniché operare secondo una logica di "push" dell'informazione, il thread audio provvede esclusivamente alla scrittura dell'evento nei registri atomici. È poi dell'Editor il compito di estrapolare i dati, interrogando in modo ciclico il processore attraverso l'impiego di un `juce::Timer` configurato a 60 Hz:

```
// Override del Callback del timer come specificato in PluginEditor.h
// Controllo la corda suonata da tastiera MIDI
void StringUIDemoAudioProcessorEditor::timerCallback()
{
    // Controllo per ogni corda se la rispettiva flag è stata alzata dall'Audio Thread (processBlock)
    // Tutto tramite Polling dell'atomic <bool> uiStringWasPlucked[numStrings]
    for (int i = 0; i < StringUIDemoAudioProcessor::numStrings; ++i)
    {
        // Check della flag, utilizzando il valore attuale e portandola a false
        if (audioProcessor.uiStringWasPlucked[i].exchange(false))
        {
            float position = audioProcessor.uiPluckPosition[i].load(); // Posizione del pizzico
            stringComponents.getUnchecked(i)->stringPlucked(position); // Aggiorna la visualizzazione della corda
            Aggiornamento label nota suonata
        }
    }
}
```

L'efficacia di tale architettura si fonda sull'impiego della procedura `exchange(false)`. Questa istruzione atomica permette di acquisire lo stato della flag e di effettuare il ripristino a `false`. Grazie a questo meccanismo, il sistema assicura una risposta immediata agli eventi MIDI, preservando l'integrità e la priorità dell'Audio Thread.

Interfaccia Grafica e User Experience

Animazione Fisica delle Corde (Visual Karplus-Strong):

Per fornire un feedback visivo della corda suonata, è stata creata una classe dedicata `StringComponent`. Invece di renderizzare l'onda simulando decine di punti indipendenti (operazione molto costosa per la GPU/CPU grafica), si è optato per un'approssimazione geometrica ottimizzata basata sulle curve di Bézier.

```
/// <summary>
/// Simulo pizzichio corda.
/// <br>pizzichio al centro = vibrazione massima.</br>
/// <br>pizzichio vicino ai bordi = vibrazione minima.</br>
/// </summary>
void stringPlucked(float pluckPositionRelative)
{
    //pluckPositionRelative è un valore tra 0 e 1 della posizione nella corda
    //calcolo l'ampiezza della vibrazione
    amplitude = maxAmplitude * std::sin(pluckPositionRelative * juce::MathConstants<Float>::pi);
    //imposta la fase nel punto corretto (sinusoide sincronizzata)
    phase = juce::MathConstants<Float>::pi;
}
```

Quando la corda viene suonata (dal click del mouse o dalla tastiera MIDI), calcola l'ampiezza dell'oscillazione in modo fisicamente coerente: pizzicando al centro (0.5) il seno restituisce 1, generando la massima vibrazione visiva. Per il disegno (`generateStringPath`), viene tracciata una curva quadratica (`quadraticTo`) il cui "ventre" centrale si sposta su e giù nel tempo seguendo la variabile `phase`. Nel metodo `timerCallback()`, l'ampiezza viene moltiplicata per un fattore di decadimento (`amplitude *= 0.99f`), permettendo all'animazione di smorzarsi fluidamente in sincronia con l'audio.

```
//curva quadratica che si muove dall'alto al basso così da dare l'effetto del movimento
p.quadraticTo(w / 2.0f, y + std::sin(phase) * amplitude, w, y);
```

Sistema di Preset:

Per permettere all'utente la memorizzazione dei suoni è stato implementato un sistema di Preset.

```
#pragma region Funzione per preset
void StringUIdemoAudioProcessorEditor::applicaPreset(int presetId)
{
    // Creiamo una funzione Lambda interna per cambiare i parametri in una riga
    auto setParam = [this](const juce::String& id, float veroValore)
    {
        if (auto* param = audioProcessor.apvts.getParameter(id))
        {
            // Convertiamo il valore reale in un valore 0-1 e lo inviamo
            param->setValueNotifyingHost(param->convertTo8to1(veroValore));
        }
    };

    // Cambia l'accordatura di tutte le 6 corde
    // Prende un array di 6 note MIDI (dalla più bassa alla più alta)
    auto setTuning = [this](int n0, int n1, int n2, int n3, int n4, int n5)
    {
        audioProcessor.setStringMidiNote(0, n0);
        audioProcessor.setStringMidiNote(1, n1);
        audioProcessor.setStringMidiNote(2, n2);
        audioProcessor.setStringMidiNote(3, n3);
        audioProcessor.setStringMidiNote(4, n4);
        audioProcessor.setStringMidiNote(5, n5);
        updateAllTuningLabels(); // Aggiorna graficamente i bottoni
    };

    // Usiamo lo switch per applicare il preset scelto
    switch (presetId)
```

La gestione dei preset sfrutta un'espressione lambda `setParam` che normalizza i valori matematici dell'interfaccia (es. 0.0 - 100.0) nel range standard dell'APVTS (0.0 - 1.0). Invocando `setValueNotifyingHost`, l'Editor comunica alla DAW non solo il cambio visivo della manopola, ma registra correttamente l'automazione.

Scalabilità Dinamica:

Infine, per garantire la compatibilità con monitor di diverse risoluzioni, tutto il posizionamento degli elementi grafici all'interno del metodo `resized()` non utilizza coordinate assolute (hardcoded), ma viene calcolato tramite un fattore di scala globale (`float scale = (float)getWidth() / 750.0f;`) e le logiche dei rettangoli di JUCE (`removeFromTop`, `reduced`, ecc.), rendendo il plugin responsivo.